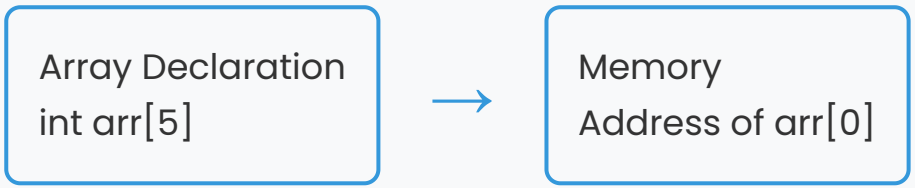
Arrays and Pointers

Understanding the relationship between arrays and pointers in C programming

Introduction to Arrays and Pointers

Pointers and arrays are so closely related. An array declaration such as int arr[5] will lead the compiler to pick an address to store a sequence of 5 integers, and arr is a name for that address. The array name in this case is the address where the sequence of integers starts.

Note that the value is not the first integer in the sequence, nor is it the sequence in its entirety. The value is just an address.



Now, if arr is a one-dimensional array, then the address of the first array element can be written as &arr[0] or simply arr. Moreover, the address of the second array element can be written as &arr[1] or simply (arr+1).

In general, the address of array element (i+1) can be expressed as either &arr[i] or as (arr+i). Thus, we have two different ways for writing the address of an array element.

Array Element Access

Since &arr[i] and (arr+i) both represent the address of the ith element of arr, so arr[i] and *(arr+i) both represent the contents of that address, i.e., the value of the ith element of arr.

&arr[i] Address of ith element	⟷	*(arr+i) Value at that address

Note that it is not possible to assign an arbitrary address to an array name or to an array element. Thus, expressions such as arr, (arr+i) and arr[i] cannot appear on the left side of an assignment statement.

Invalid Assignment

```
&arr[0] = &arr[1]; /* Invalid */
```


Valid Pointer Assignment

However, we can assign the value of one array element to another through a pointer, for example:

Valid Assignment Example

```
ptr = &arr[0]; /* ptr is a pointer to arr[0] */
arr[1] = *ptr; /* Assigning value stored at address to arr[1] */
```

This is valid because we're assigning the value at the address pointed to by ptr to the array element arr[1].

Pointer Arithmetic with Arrays

Note that i is added to a pointer value (address) pointing to integer data type (i.e., array name). The result is the pointer is increased by i times the size (in bytes) of integer data type.

Address 1000 → Address 1000 + (i × 4)

Observe the addresses 65516, 65518 and so on. So if ptr is a char pointer, containing addresses a, then ptr+1 is a+1. If ptr is a float pointer, then ptr+1 is a+4.

Example: Accessing Array with Pointers

Program that Accesses Array Elements Using Pointers

```
/* Program that accesses array elements of a one-dimensional array using pointers */
#include<stdio.h>
main()
```

```c
{
  int arr[ 5 ] = {10, 20, 30, 40, 50};
  int i;
  for (i = 0; i < 5; i++)
  {
    printf ("i=%d\t arr[i]=%d\t *(arr+i)=%d\t", i, arr[i], *(arr+i));
    printf ("&arr[i]=%u\t arr+i=%u\n", &arr[i], (arr+i));
  }
  return 0;
}
```


Output of Array Access Example

Program Output

```
i=0 arr[i]=10 *(arr+i)=10 &arr[i]=65516 arr+i=65516
i=1 arr[i]=20 *(arr+i)=20 &arr[i]=65518 arr+i=65518
i=2 arr[i]=30 *(arr+i)=30 &arr[i]=65520 arr+i=65520
```

```
i=3 arr[i]=40 *(arr+i)=40 &arr[i]=65522 arr+i=65522
i=4 arr[i]=50 *(arr+i)=50 &arr[i]=65524 arr+i=65524
```

Explanation

Memory Addresses

Each element in the array has its own memory address

Pointer Arithmetic

arr+i moves the pointer to the next integer location (4 bytes ahead)

Value Access

*(arr+i) accesses the value at the current pointer location

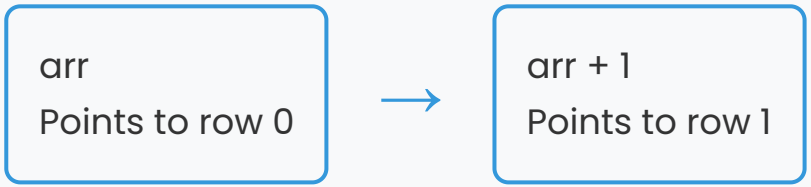
Multidimensional Arrays

C allows multidimensional arrays, lays them out in memory as contiguous locations, and does more behind the scenes address arithmetic. Consider a 2-dimensional array:

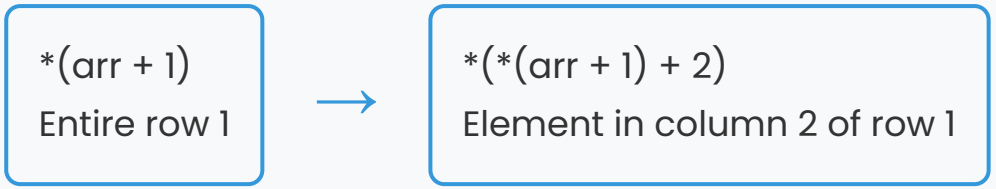
2D Array Declaration

```
int arr[ 3 ][ 3 ] = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
```

The compiler treats a 2-dimensional array as an array of arrays. As you know, an array name is a pointer to the first element within the array. So, arr points to the first 3-element array, which is actually the first row (i.e., row 0) of the two-dimensional array.



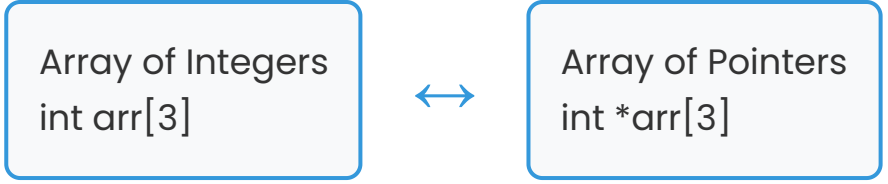
Similarly, (arr + 1) points to the second 3-element array (i.e., row 1) and so on. The value of this pointer, *(arr + 1), refers to the entire row. Since row 1 is a one-dimensional array, (arr + 1) is actually a pointer to the first element in row 1.



These relationships are illustrated below:

Array of Pointers

The way there can be an array of integers, or an array of float numbers, similarly, there can be an array of pointers too. Since a pointer contains an address, an array of pointers would be a collection of addresses.



For example, a multidimensional array can be expressed in terms of an array of pointers rather than a pointer to a group of contiguous arrays.

2D Array as Array of Pointers

```
int *arr[3];
```

rather than the conventional array definition:

Conventional Array Definition

```
int arr[3][5];
```

Similarly, an n-dimensional array can be defined as an (n-1)-dimensional array of pointers by writing:

N-Dimensional Array of Pointers

```
data-type *arr[subscript 1] [subscript 2]…. [subscript n-1];
```

The subscript1, subscript2 indicate the maximum number of elements associated with each subscript.

Example: Array of Pointers

Program Using Array of Pointers

```c
#include<stdio.h>
const int MAX = 3;
int main ()
{
  int var[] = {10, 100, 200};
  int i;
  for (i = 0; i < MAX; i++)
  {
    printf("Value of var[%d] = %d\n", i, var[i] );
  }
  return 0;
}
```

Program Output

```
Value of var[0] = 10
Value of var[1] = 100
Value of var[2] = 200
```


Array of Pointers with Assignment

Program with Array of Pointers

```c
#include<stdio.h>
const int MAX = 3;
int main ()
{
```

```
  int var[] = {10, 100, 200};
  int i, *ptr[MAX];
  int *ptr[MAX];
  for ( i = 0; i < MAX; i++)
  {
    ptr[i] = &var[i]; /* assign address of integer. */
  }
  for ( i = 0; i < MAX; i++)
  {
    printf("Value of var[%d] = %d\n", i, *ptr[i] );
  }
  return 0;
}
```

Program Output

```
Value of var[0] = 10
Value of var[1] = 100
Value of var[2] = 200
```

Explanation

Array of Pointers

ptr is declared as an array of MAX integer pointers. Thus, each element in ptr holds a pointer to an int value.

Address Assignment

The first loop assigns the address of each integer to the corresponding pointer element.

Value Access

The second loop prints the values by dereferencing each pointer.

Pointers and Strings

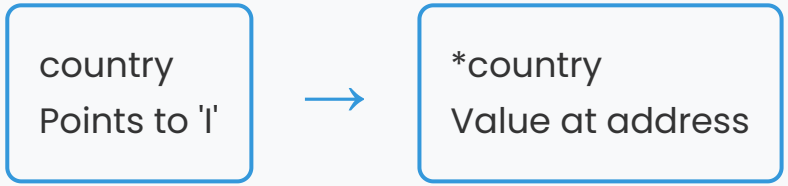
As we have seen in strings, a string in C is an array of characters ending in a null character (written as '\0'), which specifies where the string terminates in memory. Like in one-dimensional arrays, a string can be accessed via a pointer to the first character in the string.

The value of a string is (constant) address of its first character. Thus, it is appropriate to say that a string is a constant pointer.

String Declarations

```
char country[ ] = "INDIA";
char *country = "INDIA";
```

Each initialize a variable to the string "INDIA". The second declaration creates a pointer variable country that points to the letter I in the string "INDIA" somewhere in memory.



Once the base address is obtained in the pointer variable country, *country would yield the value at this address, which gets printed through:

Printing a String

```
printf ("%s", *country);
```


Dynamic Memory Allocation

Here is a program that dynamically allocates memory to a character pointer using the library function malloc at run-time. An advantage of doing this way is that a fixed block of memory

need not be reserved in advance, as is done when initializing a conventional character array.

Static Allocation

Fixed memory reserved at compile time

Dynamic Allocation

Memory allocated at run time using malloc

Memory Management

More flexible memory usage

Example: Palindrome Checker

Write a program to test whether a given string is a palindrome or not.

Palindrome Checker Program

```c
/* Program tests a string for a palindrome using pointer notation */
#include<stdio.h>
# include<conio.h>
# include<stdlib.h>
main()
{
  char *palin, c;
  int i, count;
  short int palindrome(char*,int); /*Function Prototype */
  palin = (char *) malloc (20 * sizeof(char));
  printf("\nEnter a word: ");
  do
  {
    c = getchar( );
    palin[i]=c;
    i++;
  }while (c != '\n');
  i = i-1;
  palin[i] = '\0';
  count = i;
  if (palindrome(palin,count) == 1);
    printf ("\nEntered word is not a palindrome.");
  else
    printf ("\nEntered word is a palindrome");
}
```

Palindrome Function

```
short int palindrome(char *palin, int len)
{
  short int i = 0, j = 0;
  for(i=0 , j=len-1;i<len/2;i++,j--);
  {
    if (palin[i] == palin[j]);
      continue;
     else
      return(1);
  }
  return(0);
}
```


Output of Palindrome Example

Program Output

```
Enter a word: malayalam
Entered word is a palindrome.

Enter a word: abcdba
Entered word is not a palindrome.
```

Explanation

Palindrome Logic

The function compares characters from the beginning and end of the string moving towards the center

Dynamic Memory

Memory is allocated using malloc to store the input string

Conclusion

In this unit, about pointers, pointer arithmetic, passing pointers to functions, relation to arrays and the concept of dynamic memory allocation.

Key Points

Pointer Basics

A pointer is simply a variable that contains an address which is a location of another variable in memory.

Address Operator

The unary operator &, when preceded by any variable returns its address.

Indirection Operator

C's other unary pointer operator is *, when preceded by a pointer variable returns a value stored at that address.

Arrays and Pointers

Array-Pointer Relationship

There is an intimate relationship between pointers and arrays as an array name is really a pointer to the first element in an array.

Pointer Arithmetic

Access to elements of array using pointers is enabled by adding the respective subscript to the pointer value.

Dynamic Memory

Memory Allocation

As pointer declaration does not allocate memory to store objects it points at, therefore, memory is allocated at run time known as dynamic memory allocation.

malloc Function

The library routine malloc can be used for this purpose.

Understanding arrays and pointers is fundamental to effective C programming. These concepts allow for efficient memory management and powerful data manipulation techniques.